



## LeetCode 509: Fibonacci Number

### 1. Problem Title & Link

**Title:** LeetCode 509: Fibonacci Number

**Link:** <https://leetcode.com/problems/fibonacci-number/>

### 2. Problem Statement (Short Summary)

The Fibonacci sequence is defined as:

$$F(0) = 0$$

$$F(1) = 1$$

$$F(n) = F(n-1) + F(n-2)$$

for:

$$n > 1$$

Given an integer  $n$ , return:

$$F(n)$$

#### Example

Input:

$$n = 4$$

Sequence:

0 1 1 2 3

Output:

3

because:

$$F(4) = 3$$

### 3. Examples (Input → Output)

#### Example 1

Input:

$$n = 2$$

Output:

1

Explanation:

$$F(2)$$

$$= F(1) + F(0)$$



= 1+0

= 1

### Example 2

Input:

n = 3

Output:

2

Explanation:

1+1

### Example 3

Input:

n = 4

Output:

3

### Example 4

Input:

n = 6

Output:

8

Sequence:

0 1 1 2 3 5 8

## 4. Constraints

$0 \leq n \leq 30$

Important:

Small constraint

but interview focus is

Recursion vs DP.

## 5. Core Concept (Pattern / Topic)

### Dynamic Programming

Secondary Topics:



- Recursion
- Memoization
- Iteration

## 6. Thought Process (Step-by-Step Explanation)

### Understanding Fibonacci

Given:

$F(0)=0$

$F(1)=1$

Every next value:

Previous Two Values Sum

Example:

0

1

1

2

3

5

8

13

21

...

### Recursive Thinking

To calculate:

$F(5)$

Need:

$F(4)$

+

$F(3)$

To calculate:

$F(4)$

Need:

$F(3)$

+

$F(2)$



Notice:

$F(3)$

is calculated multiple times.

### Problem

Lots of repeated work.

Example:

$F(5)$

└─  $F(4)$

| └─  $F(3)$

| └─  $F(2)$

|

└─  $F(3)$

Same:

$F(3)$

appears repeatedly.

### Better Idea

Store previous answers.

This is:

Dynamic Programming

### Best Observation

To calculate:

$F(n)$

we only need:

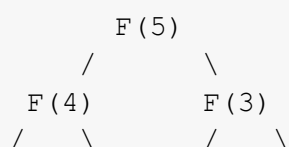
$F(n-1)$

$F(n-2)$

So keep only two variables.

### 7. Visual / Intuition Diagram (ASCII Diagram)

Recursive Tree:





```
F(3) F(2) F(2) F(1)
```

Notice:

```
F(3)
```

```
F(2)
```

repeated.

Iterative View:

```
0 1
```

```
↓
```

```
1
```

```
↓
```

```
2
```

```
↓
```

```
3
```

```
↓
```

```
5
```

Keep building next number.

## 8. Pseudocode (Language Independent)

```
if n <= 1
    return n

prev2 = 0
prev1 = 1

for i from 2 to n
    current = prev1 + prev2
    prev2 = prev1
    prev1 = current

return prev1
```

## 9. Code Implementation

### Python



```
class Solution:
    def fib(self, n: int) -> int:

        if n <= 1:
            return n

        prev2 = 0
        prev1 = 1

        for _ in range(2, n + 1):

            current = prev1 + prev2

            prev2 = prev1
            prev1 = current

        return prev1
```

## Java

```
class Solution {

    public int fib(int n) {

        if(n <= 1)
            return n;

        int prev2 = 0;
        int prev1 = 1;

        for(int i = 2; i <= n; i++) {

            int current = prev1 + prev2;

            prev2 = prev1;
            prev1 = current;
        }

        return prev1;
    }
}
```

## 10. Time & Space Complexity

### Iterative DP

Time Complexity:  $O(n)$

One loop.



Space Complexity:  $O(1)$

Only three variables.

## 11. Common Mistakes / Edge Cases

### Mistake 1

Wrong base case.

Correct:

$F(0)=0$

$F(1)=1$

### Mistake 2

Starting loop from wrong index.

### Mistake 3

Updating variables in wrong order.

## Edge Cases

**$n = 0$**

Output:

0

**$n = 1$**

Output:

1

**$n = 2$**

Output:

1

## 12. Detailed Dry Run

Input:

$n = 6$

Initial:

$prev2 = 0$

$prev1 = 1$



| i | current | prev<br>2 | prev<br>1 |
|---|---------|-----------|-----------|
| 2 | 1       | 1         | 1         |
| 3 | 2       | 1         | 2         |
| 4 | 3       | 2         | 3         |
| 5 | 5       | 3         | 5         |
| 6 | 8       | 5         | 8         |

Return:

8

### 13. Common Use Cases (Real-Life / Interview)

#### Population Growth Models

Simplified growth simulation.

#### Dynamic Programming Introduction

Classic DP starter problem.

#### Recurrence Relations

Mathematical modeling.

#### Interview Purpose

Tests:

Recursion

Memoization

Bottom-Up DP

Space Optimization

### 14. Common Traps (Important!)

#### Trap 1

Using naive recursion.

Complexity:

$O(2^n)$

Very slow.

#### Trap 2

Not recognizing overlapping subproblems.



**Trap 3**

Thinking DP always requires arrays.

Here:

$O(1)$

space is enough.

**15. Builds To (Related LeetCode Problems)**

LC 509 - Fibonacci Number



LC 70 - Climbing Stairs



LC 746 - Min Cost Climbing Stairs



LC 198 - House Robber



LC 62 - Unique Paths



LC 322 - Coin Change

This is the foundation of Dynamic Programming.

**16. Alternate Approaches + Comparison**

| Approach     | Time     | Space  | Interview Rating |
|--------------|----------|--------|------------------|
| Recursion    | $O(2^n)$ | $O(n)$ | Poor             |
| Memoization  | $O(n)$   | $O(n)$ | Good             |
| DP Array     | $O(n)$   | $O(n)$ | Good             |
| Iterative DP | $O(n)$   | $O(1)$ | Best             |

**Approach 1: Recursion**

```
def fib(n):
```

```
    if n <= 1:
```

```
        return n
```

```
    return fib(n-1) + fib(n-2)
```

Simple but inefficient.

**Approach 2: Memoization**

Store computed results.

**Approach 3: Iterative DP (Chosen)**

Most efficient.

**17. Why This Solution Works (Short Intuition)**

Each Fibonacci number depends only on the previous two Fibonacci numbers.

Instead of storing the entire sequence, we continuously update two variables representing the last two computed values.

**18. Variations / Follow-Up Questions****Follow-Up 1**

Print entire Fibonacci series up to N.

**Follow-Up 2**

Find Fibonacci using recursion + memoization.

**Follow-Up 3**

Find Fibonacci using matrix exponentiation.

Complexity:

$O(\log n)$

Advanced.

**Follow-Up 4**

Solve:

Climbing Stairs

using Fibonacci relation.

**Interview Takeaway**

This is one of the most important beginner Dynamic Programming problems.

Core DP idea:

Current Answer

=

Previous Answers

Fibonacci teaches:

Recursion



Memoization

Bottom-Up DP

Space Optimization

which appear repeatedly in medium and hard interview questions.